

|                 |                                             |
|-----------------|---------------------------------------------|
| Student Name    | Balaji MJ                                   |
| Register Number | TU-6253210211004                            |
| Project Title   | Product Review and Rating Management System |
| Week            | Week 6                                      |

## WEEK-6 PROJECT

### Task Name:

Product Review and Rating Management System

### Description:

This Week-6 task focuses on designing and implementing a Product Review and Rating Management System for an E-Commerce database. The objective is to store customer feedback and product ratings in a structured relational database and retrieve useful review information using SQL queries.

The project demonstrates how customer reviews can be stored, searched, summarised, and analysed. Students will create Review and Rating-related tables, store customer feedback, retrieve product review details, calculate average product ratings using aggregate functions, and identify highly rated products.

### 1. PROJECT OBJECTIVE

The main objective is to build a database component that manages customer opinions about products. The system should connect customers and products with their reviews and ratings, while allowing administrators or analysts to generate useful rating-based reports.

### 2. CORE REQUIREMENTS

- Create Review and Rating tables.
- Store customer feedback and ratings.
- Retrieve product review details.
- Calculate average product ratings using aggregate functions.
- Identify highly rated products.

**Example:** A customer can submit a review for a Wireless Headphones product with a rating of 5 and feedback such as "Good sound quality and comfortable to use."

### 3. DATABASE DESIGN

The review system should be connected to the existing E-Commerce entities. A review normally belongs to a customer and a product. The rating can be stored as a numeric value, while the review text stores the customer's written feedback.

| Table     | Important Columns                                             | Purpose                     |
|-----------|---------------------------------------------------------------|-----------------------------|
| Customers | customer_id, customer_name, email                             | Stores customer information |
| Products  | product_id, product_name, price                               | Stores product information  |
| Reviews   | review_id, customer_id, product_id, review_text, review_date  | Stores customer feedback    |
| Ratings   | rating_id, customer_id, product_id, rating_value, rating_date | Stores numerical ratings    |

#### Sample review records

| review_id | customer_id | product_id | rating | review_text       |
|-----------|-------------|------------|--------|-------------------|
| R101      | C101        | P101       | 5      | Excellent product |
| R102      | C102        | P101       | 4      | Good quality      |
| R103      | C103        | P102       | 5      | Very useful       |
| R104      | C104        | P102       | 3      | Average product   |

#### Rating range

For a standard five-point rating system, rating values can be constrained to an appropriate range such as 1 to 5. The database should validate rating values so that invalid scores are not inserted.

---

## 4. CREATE REVIEW AND RATING TABLES

The first implementation step is to create the tables required to store review and rating information. Primary keys uniquely identify records, while foreign keys connect reviews and ratings to customers and products.

### Reviews table

```
CREATE TABLE Reviews (  
    review_id INT PRIMARY KEY,  
    customer_id INT,  
    product_id INT,  
    review_text VARCHAR(500),  
    review_date DATE,  
    FOREIGN KEY (customer_id) REFERENCES Customers(customer_id),  
    FOREIGN KEY (product_id) REFERENCES Products(product_id)  
);
```

### Ratings table

```
CREATE TABLE Ratings (  
    rating_id INT PRIMARY KEY,  
    customer_id INT,  
    product_id INT,  
    rating_value INT,  
    rating_date DATE,  
    FOREIGN KEY (customer_id) REFERENCES Customers(customer_id),  
    FOREIGN KEY (product_id) REFERENCES Products(product_id)  
);
```

If the selected DBMS supports CHECK constraints, the rating field can also be restricted to valid values.

```
CHECK (rating_value BETWEEN 1 AND 5)
```

## 5. STORE CUSTOMER FEEDBACK AND RATINGS

Example review insertion:

```
INSERT INTO Reviews  
(review_id, customer_id, product_id, review_text, review_date)  
VALUES  
(101, 1, 101, 'Excellent product and good quality', '2026-09-20');
```

Example rating insertion:

```
INSERT INTO Ratings  
(rating_id, customer_id, product_id, rating_value, rating_date)  
VALUES  
(101, 1, 101, 5, '2026-09-20');
```

---

## 6. RETRIEVE PRODUCT REVIEW DETAILS

SQL SELECT queries can be used to retrieve customer feedback and rating information for a particular product.

### Retrieve all reviews

```
SELECT review_id, customer_id, product_id,  
       review_text, review_date  
FROM Reviews;
```

### Retrieve reviews for a particular product

```
SELECT review_id, customer_id, review_text, review_date  
FROM Reviews  
WHERE product_id = 101;
```

### Retrieve high-rated reviews

```
SELECT review_id, product_id, rating_value  
FROM Ratings  
WHERE rating_value >= 4;
```

### Retrieve product reviews with customer details

```
SELECT r.review_id,  
       r.product_id,  
       r.customer_id,  
       r.review_text,  
       r.review_date  
FROM Reviews r  
WHERE r.product_id = 101  
ORDER BY r.review_date DESC;
```

### Business use

- Display reviews on a product page.
- Allow administrators to inspect customer feedback.
- Find highly positive or negative reviews.
- Track recent customer feedback.

---

## 7. CALCULATE AVERAGE PRODUCT RATINGS

Aggregate functions can summarise rating data. AVG() is used to calculate the average rating of a product.

### Average rating for one product

```
SELECT product_id,  
       AVG(rating_value) AS average_rating  
FROM Ratings  
WHERE product_id = 101  
GROUP BY product_id;
```

### Average rating for every product

```
SELECT product_id,  
       AVG(rating_value) AS average_rating  
FROM Ratings  
GROUP BY product_id;
```

### Count the number of ratings

```
SELECT product_id,  
       COUNT(*) AS total_ratings  
FROM Ratings  
GROUP BY product_id;
```

### Average rating with number of reviews

```
SELECT product_id,  
       COUNT(*) AS total_ratings,  
       AVG(rating_value) AS average_rating  
FROM Ratings  
GROUP BY product_id;
```

### Example calculation

Suppose a product receives ratings of 5, 4, 5, and 3. The average rating is:

$$(5 + 4 + 5 + 3) / 4 = 4.25$$

The actual project should calculate averages directly from the database rather than relying on manually calculated values.

---

## 8. IDENTIFY HIGHLY RATED PRODUCTS

After calculating average ratings, the system can identify products whose ratings meet a defined criterion. For example, the project can identify products with an average rating of at least 4.0.

### Highly rated products

```
SELECT product_id,  
       AVG(rating_value) AS average_rating  
FROM Ratings  
GROUP BY product_id  
HAVING AVG(rating_value) >= 4.0  
ORDER BY average_rating DESC;
```

### Highly rated products with minimum review count

```
SELECT product_id,  
       COUNT(*) AS total_ratings,  
       AVG(rating_value) AS average_rating  
FROM Ratings  
GROUP BY product_id  
HAVING AVG(rating_value) >= 4.0  
       AND COUNT(*) >= 3  
ORDER BY average_rating DESC;
```

Using a minimum number of ratings can make a report more informative because a product with one five-star rating does not have the same amount of review evidence as a product with many ratings.

## 9. BASIC REVIEW AND RATING REPORT

| Product | Total Ratings | Average Rating | Status       |
|---------|---------------|----------------|--------------|
| P101    | 25            | 4.52           | Highly Rated |
| P102    | 18            | 4.17           | Highly Rated |
| P103    | 11            | 3.64           | Regular      |
| P104    | 7             | 2.86           | Low Rated    |

The values above are illustrative only and should be replaced by the actual query results generated from the project database.

## 10. END-TO-END SYSTEM WORKFLOW

The complete Product Review and Rating Management System should follow a simple database workflow:

|                                   |
|-----------------------------------|
| Customer and Product Data         |
| ↓                                 |
| Create Reviews and Ratings Tables |
| ↓                                 |
| Insert Customer Feedback          |
| ↓                                 |
| Store Numerical Ratings           |
| ↓                                 |
| Retrieve Review Details           |
| ↓                                 |
| Calculate Rating Statistics       |
| ↓                                 |
| Calculate Average Ratings         |
| ↓                                 |
| Filter Highly Rated Products      |
| ↓                                 |
| Generate Review and Rating Report |

## 11. RECOMMENDED TECHNOLOGIES

- MySQL, PostgreSQL, Oracle SQL, or SQLite.
- MySQL Workbench, pgAdmin, Oracle SQL Developer, or another SQL client.
- A relational database for storing customer, product, review, and rating information.
- Optional application frontend for displaying product reviews and ratings.

### Important SQL concepts

- CREATE TABLE
- INSERT INTO
- SELECT
- WHERE
- GROUP BY
- HAVING
- ORDER BY
- COUNT()
- AVG()

## 12. VALIDATION AND TESTING

The project should test both data integrity and query correctness. Review and rating records should be connected to valid customers and products.

- Verify that every review has a valid customer and product reference.
- Verify that rating values remain within the selected rating range.
- Check that product review queries return only the requested product.
- Verify AVG() results against sample data.
- Check that GROUP BY produces one summary row per product.
- Verify HAVING filters products according to the average-rating condition.
- Check descending ORDER BY results for highly rated products.

### Example test cases

| Test                         | Expected behaviour                           |
|------------------------------|----------------------------------------------|
| product_id = 101             | Only reviews for product 101                 |
| rating_value >= 4            | Ratings of 4 and 5                           |
| AVG(rating_value)            | Correct average per product                  |
| HAVING AVG(...) >= 4         | Only products meeting the selected threshold |
| ORDER BY average_rating DESC | Highest average appears first                |

## 13. FINAL PROJECT DELIVERABLES

1. Review table design.
2. Rating table design.
3. Sample customer feedback records.
4. Sample numerical rating records.
5. SQL queries for retrieving product review details.
6. Average rating calculation using AVG().
7. Rating count using COUNT().
8. Queries for identifying highly rated products.
9. Review and rating summary report.
10. Validation and testing results.

---

## 14. PROJECT OBJECTIVE AND FINAL OUTPUT

The main objective of this project is to implement a database-driven system for managing customer reviews and product ratings. The system should provide a structured way to store feedback, retrieve review information, calculate product rating statistics, and identify highly rated products using SQL aggregate and filtering operations.

### FINAL OUTPUT

| Output                | Description                                            |
|-----------------------|--------------------------------------------------------|
| Review Details        | Customer feedback associated with products             |
| Rating Details        | Numerical rating given by customers                    |
| Average Rating        | Average rating calculated for each product             |
| Rating Count          | Number of ratings received by each product             |
| Highly Rated Products | Products meeting the selected average-rating criterion |
| Report                | Summary of product review and rating performance       |

**Note:** Sample records and numerical results in this document are illustrative. The final values should be generated from the actual database and SQL queries created for the project.